

## TOPIC 1 : INTRODUCTION

**1.** Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward. **Example 1:** Input: words = ["abc","car","ada","racecar","cool"] Output: "ada" **Explanation:** The first string that is palindromic is "ada". Note that "racecar" is also palindromic, but it is not the first. **Example 2:** Input: words = ["notapalindrome","racecar"] Output: "racecar" **Explanation:** The first and only string that is palindromic is "racecar".

### Aim:

To write a C program to find and return the **first palindromic string** in an array of strings. If no palindrome is found, display an empty string.

### Algorithm:

1. Start the program.
2. Read the number of strings (n).
3. Read all the strings into an array.
4. Traverse each string one by one.
5. Check whether the current string is a palindrome by comparing characters from the beginning and end.
6. If a palindrome is found:
  - Display the string.
  - Stop the search.
7. If no palindrome is found after checking all strings:
  - Display an empty string ("").
8. End the program.

### Source Code:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int isPalindrome(char str[])
5 {
6     int left = 0;
7     int right = strlen(str) - 1;
8
9     while (left < right)
10    {
11        if (str[left] != str[right])
12            return 0;
13
14        left++;
15        right--;
16    }
17
18    return 1;
```

```

19 }
20
21 int main()
22 {
23     int n, i;
24     char words[100][100];
25     int found = 0;
26
27     printf("Enter number of words: ");
28     scanf("%d", &n);
29
30     printf("Enter the words:\n");
31
32     for(i = 0; i < n; i++)
33     {
34         scanf("%s", words[i]);
35     }
36

```

```

36     for(i = 0; i < n; i++)
37     {
38         if(isPalindrome(words[i]))
39         {
40             printf("\nFirst Palindromic String: %s\n", words[i]);
41             found = 1;
42             break;
43         }
44     }
45
46     if(found == 0)
47     {
48         printf("\nFirst Palindromic String: \n\n");
49     }
50
51     return 0;
52 }
53

```

### Sample input and output:

```

Enter number of words: 5
Enter the words:
abc
car
ada
racecar
cool

First Palindromic String: ada

```

```

Enter number of words: 2
Enter the words:
notapalindrome
racecar

First Palindromic String: racecar

```

### Time Complexity:

- Let **N** = Number of strings.
- Let **M** = Maximum length of a string.

Checking one string takes **O(M)** time.

Checking all strings takes:

**Time Complexity = O(N × M)**

### Space Complexity:

The algorithm uses only a few extra variables for comparison.

**Space Complexity = O(1)** (excluding the input storage)

**Result:**

Thus, the C program to find the **first palindromic string** in an array of strings was implemented successfully. The program correctly displays the first palindrome if one exists; otherwise, it displays an empty string ("").

2. You are given two integer arrays `nums1` and `nums2` of sizes `n` and `m`, respectively. Calculate the following values: `answer1` : the number of indices `i` such that `nums1[i]` exists in `nums2`. `answer2` : the number of indices `i` such that `nums2[i]` exists in `nums1`. Return `[answer1,answer2]`.  
**Example 1:** Input: `nums1 = [2,3,2]`, `nums2 = [1,2]` Output: `[2,1]` Explanation: **Example 2:** Input: `nums1 = [4,3,2,3,1]`, `nums2 = [2,2,5,2,3,6]` Output: `[3,4]` Explanation: The elements at indices 1, 2, and 3 in `nums1` exist in `nums2` as well. So `answer1` is 3. The elements at indices 0, 1, 3, and 4 in `nums2` exist in `nums1`. So `answer2` is 4.

### Aim:

To write a C program to find:

- **answer1:** The number of elements in `nums1` that also exist in `nums2`.
- **answer2:** The number of elements in `nums2` that also exist in `nums1`.

Finally, display the result as `[answer1, answer2]`.

### Algorithm:

1. Start the program.
2. Read the size of the first array (`n`) and its elements.
3. Read the size of the second array (`m`) and its elements.
4. Initialize `answer1` and `answer2` to 0.
5. Traverse each element of `nums1`.
  - Check whether it exists in `nums2`.
  - If found, increment `answer1` and move to the next element.
6. Traverse each element of `nums2`.
  - Check whether it exists in `nums1`.
  - If found, increment `answer2` and move to the next element.
7. Print the result in the format `[answer1, answer2]`.
8. Stop the program.

### Source Code:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, m;
6     int nums1[100], nums2[100];
7     int answer1 = 0, answer2 = 0;
8     int i, j;
9
10    printf("Enter size of first array: ");
11    scanf("%d", &n);
12
13    printf("Enter elements of first array:\n");
14    for(i = 0; i < n; i++)
15    {
16        scanf("%d", &nums1[i]);
17    }
18
```

```
    printf("Enter size of second array: ");
    scanf("%d", &m);
    printf("Enter elements of second array:\n");
    for(i = 0; i < m; i++)
    {
        scanf("%d", &nums2[i]);
    }
    // Calculate answer1
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            if(nums1[i] == nums2[j])
            {
                answer1++;
                break;
            }
        }
    }
    // Calculate answer2
    for(j = 0; j < m; j++)
    {
        for(i = 0; i < n; i++)
        {
            if(nums2[j] == nums1[i])
            {
                answer2++;
                break;
            }
        }
    }
    printf("The result is: [%d, %d]", answer1, answer2);
    return 0;
}
```

```

40
41 // Calculate answer2
42 for(i = 0; i < m; i++)
43 {
44     for(j = 0; j < n; j++)
45     {
46         if(nums2[i] == nums1[j])
47         {
48             answer2++;
49             break;
50         }
51     }
52 }
53
54 printf("\nOutput: [%d, %d]\n", answer1, answer2);
55
56 return 0;
57 }

```

### Sample input and output:

```

^ Enter size of first array: 3
Enter elements of first array:
2
3
2
Enter size of second array: 2
Enter elements of second array:
1
2
Output: [2, 1]

```

```

Enter size of first array: 5
Enter elements of first array:
4
3
2
3
1
Enter size of second array: 6
Enter elements of second array:
2
2
5
2
3
6
Output: [3, 4]

```

### Time Complexity:

- Let n be the size of nums1.
- Let m be the size of nums2.

The program compares every element of one array with the other array.

Time Complexity =  $O(n \times m)$

### Space Complexity:

The program uses only a few extra variables (answer1, answer2, i, j) apart from the input arrays.

Space Complexity =  $O(1)$  (excluding the input arrays)

### Result:

Thus, the C program to calculate the number of common elements between two arrays was implemented successfully. The program correctly counts:

- answer1: Number of elements in nums1 that exist in nums2.
- answer2: Number of elements in nums2 that exist in nums1.

The output is displayed in the format [answer1, answer2].

**3. You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let  $\text{nums}[i..j]$  be a subarray of nums consisting of all the indices from  $i$  to  $j$  such that  $0 \leq i \leq j < \text{nums.length}$ . Then the number of distinct values in  $\text{nums}[i..j]$  is called the distinct count of  $\text{nums}[i..j]$ . Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array. Example 1: Input:  $\text{nums} = [1,2,1]$  Output: 15 Explanation: Six possible subarrays are:  $[1]$ : 1 distinct value  $[2]$ : 1 distinct value  $[1]$ : 1 distinct value  $[1,2]$ : 2 distinct values  $[2,1]$ : 2 distinct values  $[1,2,1]$ : 2 distinct values The sum of the squares of the distinct counts in all subarrays is equal to  $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$ . Example 2: Input:  $\text{nums} = [1,1]$  Output: 3 Explanation: Three possible subarrays are:  $[1]$ : 1 distinct value  $[1]$ : 1 distinct value  $[1,1]$ : 1 distinct value The sum of the squares of the distinct counts in all subarrays is equal to  $1^2 + 1^2 + 1^2 = 3$ .**

### Aim:

To write a C program to calculate the sum of the squares of the distinct counts of all possible subarrays of a given integer array.

### Algorithm:

1. Start the program.
2. Read the size of the array (n) and its elements.
3. Initialize sum = 0.
4. Generate all possible subarrays using two loops:
  - o Outer loop for the starting index.
  - o Inner loop for the ending index.
5. For each subarray:
  - o Count the number of distinct elements by checking previously seen elements within the current subarray.
  - o Store the distinct count.
6. Calculate the square of the distinct count.
7. Add the squared value to sum.
8. Repeat until all subarrays are processed.
9. Display the final sum.
10. Stop the program.

### Source Code:

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6      int nums[100];
7      int i, j, k;
8      int sum = 0;
9
10     printf("Enter size of array: ");
11     scanf("%d", &n);
12
13     printf("Enter array elements:\n");
14     for(i = 0; i < n; i++)
15     {
16         scanf("%d", &nums[i]);
17     }
18

```

```

20     for(i = 0; i < n; i++)
21     {
22         for(j = i; j < n; j++)
23         {
24             int distinct = 0;
25
26             // Count distinct elements in subarray nums[i...j]
27             for(k = i; k <= j; k++)
28             {
29                 int found = 0;
30
31                 for(int l = i; l < k; l++)
32                 {
33                     if(nums[l] == nums[k])
34                     {
35                         found = 1;
36                         break;
37                     }
38                 }
39             }
40         }
41     }
42

```

```

34-         {
35-             found = 1;
36-             break;
37-         }
38-     }
39-
40-     if(found == 0)
41-         distinct++;
42- }
43-
44- sum += distinct * distinct;
45- }
46- }
47-
48- printf("\nSum of Squares of Distinct Counts = %d\n", sum);
49-
50- return 0;
51- }

```

### Sample input and output:

```

Enter size of array: 3
Enter array elements:
1
2
1

Sum of Squares of Distinct Counts = 15

```

```

Enter size of array: 3
Enter array elements:
1
2
1

Sum of Squares of Distinct Counts = 15

```

### Time Complexity:

Let **n** be the size of the array.

- Generating all subarrays takes  **$O(n^2)$** .
- Counting distinct elements in each subarray takes  **$O(n^2)$**  in the worst case.

**Overall Time Complexity =  $O(n^4)$**

### Space Complexity:

The program uses only a few extra variables for counting distinct elements.

**Space Complexity =  $O(1)$**  (excluding the input array)

### Result:

Thus, the C program to calculate the **sum of the squares of the distinct counts of all subarrays** was implemented successfully. The program correctly computes the distinct count for every subarray, squares it, adds the values, and displays the final result.

**4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where  $0 \leq i < j < n$ , such that  $\text{nums}[i] == \text{nums}[j]$  and  $(i * j)$  is divisible by k.**

**Example 1: Input: nums = [3,1,2,2,2,1,3], k = 2 Output: 4 Explanation: There are 4 pairs that meet all the requirements: -  $\text{nums}[0] == \text{nums}[6]$ , and  $0 * 6 == 0$ , which is divisible by 2. -  $\text{nums}[2] == \text{nums}[3]$ , and  $2 * 3 == 6$ , which is divisible by 2. -  $\text{nums}[2] == \text{nums}[4]$ , and  $2 * 4 == 8$ , which is divisible by 2. -  $\text{nums}[3] == \text{nums}[4]$ , and  $3 * 4 == 12$ , which is divisible by 2.**

**Example 2: Input: nums = [1,2,3,4], k = 1 Output: 0 Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.**

#### **Aim:**

To write a C program to count the number of pairs (i, j) such that:

- $0 \leq i < j < n$
- $\text{nums}[i] == \text{nums}[j]$
- $(i \times j)$  is divisible by k

#### **Algorithm:**

1. Start the program.
2. Read the size of the array (n).
3. Read the array elements.
4. Read the value of k.
5. Initialize a variable count = 0.
6. Use two nested loops:
  - Outer loop from  $i = 0$  to  $n - 2$ .
  - Inner loop from  $j = i + 1$  to  $n - 1$ .
7. For every pair (i, j):
  - Check if  $\text{nums}[i] == \text{nums}[j]$ .
  - Check if  $(i * j) \% k == 0$ .
  - If both conditions are true, increment count.
8. Display the total number of valid pairs.
9. Stop the program.



## Source Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n, k;
6      int nums[100];
7      int i, j;
8      int count = 0;
9
10     printf("Enter size of array: ");
11     scanf("%d", &n);
12
13     printf("Enter array elements:\n");
14     for(i = 0; i < n; i++)
15     {
16         scanf("%d", &nums[i]);
17     }
18
19     printf("Enter value of k: ");
20     scanf("%d", &k);
21
22     for(i = 0; i < n - 1; i++)
23     {
24         for(j = i + 1; j < n; j++)
25         {
26             if(nums[i] == nums[j] && ((i * j) % k == 0))
27             {
28                 count++;
29             }
30         }
31     }
32
33     printf("\nNumber of Valid Pairs = %d\n", count);
34
35     return 0;
36 }
```

## Sample input and output:

```
Enter size of array: 7
Enter array elements:
3 1 2 2 2 1 3
Enter value of k: 2

Number of Valid Pairs = 4
```

```
Enter size of array: 4
Enter array elements:
1 2 3 4
Enter value of k: 1

Number of Valid Pairs = 0
```

## Time Complexity:

Let **n** be the number of elements in the array.

The program checks every possible pair using two nested loops.

**Time Complexity =  $O(n^2)$**

## Space Complexity:

The program uses only a few extra variables (count, i, j) apart from the input array.

**Space Complexity =  $O(1)$**  (excluding the input array)

## Result:

Thus, the C program to count the number of pairs (**i, j**) satisfying the conditions `nums[i] == nums[j]` and `(i × j) divisible by k` was implemented successfully. The program correctly displays the total number of valid pairs.

**5. Write a program FOR THE BELOW TEST CASES with least time complexity**    **Test Cases:**  
- Input: {1, 2, 3, 4, 5} Expected Output: 5 Input: {7, 7, 7, 7, 7} Expected Output: 7 Input: {-10, 2, 3, -4, 5} Expected Output: 5

**Aim:**

To write a C program to find the maximum element in an array with the least possible time complexity.

**Algorithm:**

1. Start the program.
2. Read the size of the array (n).
3. Read the array elements.
4. Assume the first element is the maximum.
5. Traverse the array from the second element to the last.
6. If the current element is greater than the maximum, update the maximum.
7. After completing the traversal, display the maximum element.
8. Stop the program.

**Source Code:**

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i;
6     int arr[100];
7     int max;
8
9     printf("Enter the size of the array: ");
10    scanf("%d", &n);
11
12    printf("Enter the array elements:\n");
13    for(i = 0; i < n; i++)
14    {
15        scanf("%d", &arr[i]);
16    }
17
18    max = arr[0];
19
20    for(i = 1; i < n; i++)
21    {
22        if(arr[i] > max)
23        {
24            max = arr[i];
25        }
26    }
27
28    printf("\nMaximum Element = %d\n", max);
29
30    return 0;
31 }
```

**Sample input and output:**

```
Enter the size of the array: 5
Enter the array elements:
1 2 3 4 5

Maximum Element = 5
```

```
Enter the size of the array: 5
Enter the array elements:
7 7 7 7 7

Maximum Element = 7
```

```
Enter the size of the array: 5
Enter the array elements:
-10 2 3 -4 5

Maximum Element = 5
```

**Time Complexity:**

The array is traversed only once.

Time Complexity =  $O(n)$

### Space Complexity:

Only one extra variable (max) is used.

Space Complexity =  $O(1)$

### Result:

Thus, the C program to find the maximum element in an array was implemented successfully. The program correctly returns the maximum value for all the given test cases with the least possible  $O(n)$  time complexity and  $O(1)$  space complexity.

**6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same. Test Cases**

**1. Empty List** 1. Input: [] 2. Expected Output: None or an appropriate message indicating that the list is empty.

**Single Element List** 1. Input: [5] 2. Expected Output: 5

**All Elements are the Same** 1. Input: [3, 3, 3, 3, 3] 2. Expected Output: 3

### Aim

To write a C program that sorts a list of numbers using an efficient sorting algorithm and then finds the maximum element in the sorted list.

### Algorithm

1. Start the program.
2. Read the number of elements (n).
3. If  $n = 0$ , display a message indicating that the list is empty and terminate.
4. Read the array elements.
5. Sort the array in ascending order using Quick Sort.
6. After sorting, the last element of the array is the maximum element.
7. Display the sorted array.
8. Display the maximum element.
9. Stop the program.

### Source Code:

```
1  #include <stdio.h>
2
3  void quickSort(int arr[], int low, int high);
4  int partition(int arr[], int low, int high);
5
6  int partition(int arr[], int low, int high)
7  {
8      int pivot = arr[high];
9      int i = low - 1;
10     int j, temp;
11
12     for(j = low; j < high; j++)
13     {
14         if(arr[j] <= pivot)
15         {
16             i++;
17             temp = arr[i];
18             arr[i] = arr[j];
19             arr[j] = temp;
20         }
21     }
22
23     temp = arr[i + 1];
24     arr[i + 1] = arr[high];
25     arr[high] = temp;
26
27     return i + 1;
28 }
29
30 void quickSort(int arr[], int low, int high)
31 {
32     if(low < high)
33     {
34         int pi = partition(arr, low, high);
35
36         quickSort(arr, low, pi - 1);
```

```

36     quickSort(arr, low, pi - 1);
37     quickSort(arr, pi + 1, high);
38 }
39 }
40
41 int main()
42 {
43     int n, i;
44     int arr[100];
45
46     printf("Enter the number of elements: ");
47     scanf("%d", &n);
48
49     if(n == 0)
50     {
51         printf("The list is empty.\n");
52         return 0;
53     }

```

```

55     printf("Enter the array elements:\n");
56     for(i = 0; i < n; i++)
57     {
58         scanf("%d", &arr[i]);
59     }
60
61     quickSort(arr, 0, n - 1);
62
63     printf("\nSorted Array: ");
64     for(i = 0; i < n; i++)
65     {
66         printf("%d ", arr[i]);
67     }
68
69     printf("\nMaximum Element = %d\n", arr[n - 1]);
70
71     return 0;
72 }

```

### Sample input and output:

```

Enter the number of elements: 0
The list is empty.

```

```

Enter the number of elements: 5
Enter the array elements:
3 3 3 3 3

Sorted Array: 3 3 3 3 3
Maximum Element = 3

```

### Time Complexity

- Quick Sort (Average Case):  $O(n \log n)$
- Finding Maximum in Sorted Array:  $O(1)$  (last element)

Overall Time Complexity =  $O(n \log n)$

### Space Complexity

- Average Case:  $O(\log n)$  (recursive stack in Quick Sort)

### Result

Thus, the C program to sort an array using Quick Sort and find the maximum element was implemented successfully. The program correctly handles empty lists, single-element lists, and arrays with duplicate elements, displaying the sorted array and the maximum element.

**7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm? Test Cases**  
**Some Duplicate Elements** Input: [3, 7, 3, 5, 2, 5, 9, 2] Expected Output: [3, 7, 5, 2, 9]  
**(Order may vary based on the algorithm used)**  
**Negative and Positive Numbers** Input: [-1, 2, -1, 3, 2, -2] Expected Output: [-1, 2, 3, -2] **(Order may vary)**  
**List with Large Numbers** Input: [1000000, 999999, 1000000] Expected Output: [1000000, 999999]

### Aim

To write a C program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list.

## Algorithm

1. Start the program.
2. Read the number of elements (n).
3. Read the array elements.
4. Create an empty array unique[] to store unique elements.
5. Traverse each element of the original array.
6. For each element:
  - Check whether it already exists in the unique[] array.
  - If it does not exist, add it to unique[].
7. Display the unique elements.
8. Stop the program.

## Source Code:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, i, j;
6     int arr[100], unique[100];
7     int uniqueCount = 0;
8     int found;
9
10    printf("Enter the number of elements: ");
11    scanf("%d", &n);
12
13    printf("Enter the array elements:\n");
14    for(i = 0; i < n; i++)
15    {
16        scanf("%d", &arr[i]);
17    }
```

```
19    for(i = 0; i < n; i++)
20    {
21        found = 0;
22
23        for(j = 0; j < uniqueCount; j++)
24        {
25            if(arr[i] == unique[j])
26            {
27                found = 1;
28                break;
29            }
30        }
31
32        if(found == 0)
33        {
34            unique[uniqueCount] = arr[i];
35            uniqueCount++;
36        }
```

```
32    if(found == 0)
33    {
34        unique[uniqueCount] = arr[i];
35        uniqueCount++;
36    }
37
38    printf("\nUnique Elements: ");
39
40    for(i = 0; i < uniqueCount; i++)
41    {
42        printf("%d ", unique[i]);
43    }
44
45    printf("\n");
46
47    return 0;
48
49 }
```

## Sample input and output:

```
Enter the number of elements: 6
Enter the array elements:
-1 2 -1 3 2 -2

Unique Elements: -1 2 3 -2
```

```
Enter the number of elements: 3
Enter the array elements:
1000000 999999 1000000

Unique Elements: 1000000 999999
```

```
Enter the number of elements: 8
Enter the array elements:
3 7 3 5 2 5 9 2

Unique Elements: 3 7 5 2 9
```

## Time Complexity

The program compares each element with the elements already stored in the unique array.

Time Complexity =  $O(n^2)$

## Space Complexity

An additional array is used to store the unique elements.

Space Complexity =  $O(n)$

## Result

Thus, the C program to create a new list containing only the unique elements from the given array was implemented successfully. The program correctly removes duplicate elements while preserving the first occurrence of each unique value.

## 8.Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

### Aim

To write a C program to sort an array of integers using the Bubble Sort technique and analyze its time complexity using Big-O notation.

### Algorithm

1. Start the program.
2. Read the number of elements (n).
3. Read the array elements.
4. Compare each element with its adjacent element.
5. If the current element is greater than the next element, swap them.
6. Repeat the process for all elements until the array is sorted.
7. Display the sorted array.
8. Stop the program.

### Source Code:

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int arr[100], n, i, j, temp;
6
7      printf("Enter the number of elements: ");
8      scanf("%d", &n);
9
10     printf("Enter the array elements:\n");
11     for(i = 0; i < n; i++)
12     {
13         scanf("%d", &arr[i]);
14     }
15
16     // Bubble Sort
17     for(i = 0; i < n - 1; i++)
18     {
19         for(j = 0; j < n - i - 1; j++)
20         {
21             if(arr[j] > arr[j + 1])
22             {
23                 temp = arr[j];
24                 arr[j] = arr[j + 1];
25                 arr[j + 1] = temp;
26             }
27         }
28     }
29
30     printf("\nSorted Array: ");
31     for(i = 0; i < n; i++)
32     {
33         printf("%d ", arr[i]);
34     }
35
36     return 0;
```

### Sample input and output:

```
Enter the number of elements: 5
Enter the array elements:
64 34 25 12 22

Sorted Array: 12 22 25 34 64
```

## Time Complexity (Big-O Notation)

- Best Case (Already Sorted):  $O(n)$  (with an optimized Bubble Sort that checks for swaps)
- Average Case:  $O(n^2)$
- Worst Case (Reverse Sorted):  $O(n^2)$

For the above basic implementation (without optimization), the time complexity is  $O(n^2)$  in the average and worst cases.

## Space Complexity

$O(1)$  (In-place sorting; only a temporary variable is used.)

## Result

Thus, the C program to sort an array using the Bubble Sort technique was implemented successfully. The array was sorted in ascending order, and the algorithm has a time complexity of  $O(n^2)$  (average and worst case) and a space complexity of  $O(1)$ .

**9. Checks if a given number  $x$  exists in a sorted array  $arr$  using binary search. Analyze its time complexity using Big-O notation. Test Case: Example  $X=\{ 3,4,6,-9,10,8,9,30\}$  KEY=10 Output: Element 10 is found at position 5 Example  $X=\{ 3,4,6,-9,10,8,9,30\}$  KEY=100 Output : Element 100 is not found**

## Aim

To write a C program to check whether a given number (key) exists in a sorted array using the Binary Search technique and analyze its time complexity using Big-O notation.

## Algorithm

1. Start the program.
2. Read the number of elements ( $n$ ).
3. Read the sorted array elements.
4. Read the key element to be searched.
5. Set  $low = 0$  and  $high = n - 1$ .
6. Repeat while  $low \leq high$ :

- Find the middle index:

$mid = (low + high) / 2$

- If  $arr[mid] == key$ , display the position and stop.
- If  $key < arr[mid]$ , search the left half.
- Otherwise, search the right half.

7. If the key is not found, display "Element not found."

8. Stop the program.

### Source Code:

```
1 #include <stdio.h>
2 int main()
3 {
4     int arr[100], n, key;
5     int low, high, mid;
6     int i, found = 0;
7
8     printf("Enter the number of elements: ");
9     scanf("%d", &n);
10
11     printf("Enter the sorted array elements:\n");
12     for(i = 0; i < n; i++)
13     {
14         scanf("%d", &arr[i]);
15     }
16     printf("Enter the key element: ");
17     scanf("%d", &key);
18     low = 0;
19
20     while(low <= high)
21     {
22         mid = (low + high) / 2;
23
24         if(arr[mid] == key)
25         {
26             printf("Element %d is found at position %d\n", key, mid + 1);
27             found = 1;
28             break;
29         }
30         else if(key < arr[mid])
31         {
32             high = mid - 1;
33         }
34         else
35         {
36             low = mid + 1;
37         }
38     }
39     if(found == 0)
40     {
41         printf("Element %d is not found\n", key);
42     }
```

### Sample input and output:

```
Enter the number of elements: 8
Enter the sorted array elements:
-9 3 4 6 8 9 10 30
Enter the key element: 10
Element 10 is found at position 7
```

```
Enter the number of elements: 8
Enter the sorted array elements:
-9 3 4 6 8 9 10 30
Enter the key element: 100
Element 100 is not found
```

### Time Complexity (Big-O Notation)

- **Best Case:  $O(1)$**  (Key found at the middle element)
- **Average Case:  $O(\log n)$**
- **Worst Case:  $O(\log n)$**

### Space Complexity

**$O(1)$**  (Iterative implementation)

### Result

Thus, the C program to search an element in a **sorted array using Binary Search** was implemented successfully. The program correctly identifies whether the key is present in the array and displays its position. The algorithm has a **time complexity of  $O(\log n)$**  and a **space complexity of  $O(1)$** .



**10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in  $O(n \log(n))$  time complexity and with the smallest space complexity possible.**

### Aim

To write a C program to sort an array of integers in ascending order without using any built-in sorting functions, achieving  $O(n \log n)$  time complexity.

### Algorithm

1. Start the program.
2. Read the number of elements (n).
3. Read the array elements.
4. Divide the array into two halves recursively until each subarray contains one element.
5. Merge the subarrays by comparing their elements and arranging them in ascending order.
6. Repeat until the entire array is sorted.
7. Display the sorted array.
8. Stop the program.

### Source Code:

```
1 #include <stdio.h>
2 void merge(int arr[], int left, int mid, int right)
3 {
4     int i, j, k;
5     int n1 = mid - left + 1;
6     int n2 = right - mid;
7
8     int L[100], R[100];
9
10    for(i = 0; i < n1; i++)
11        L[i] = arr[left + i];
12
13    for(j = 0; j < n2; j++)
14        R[j] = arr[mid + 1 + j];
15
16    i = 0;
17    j = 0;
18    k = left;
19    while(i < n1 && j < n2)
```

```
20        if(L[i] <= R[j])
21            arr[k++] = L[i++];
22        else
23            arr[k++] = R[j++];
24    }
25    while(i < n1)
26        arr[k++] = L[i++];
27    while(j < n2)
28        arr[k++] = R[j++];
29 }
30 void mergeSort(int arr[], int left, int right)
31 {
32     if(left < right)
33     {
34         int mid = left + (right - left) / 2;
35         mergeSort(arr, left, mid);
36         mergeSort(arr, mid + 1, right);
37         merge(arr, left, mid, right);
```

```
40 int main()
41 {
42     int arr[100], n, i;
43     printf("Enter the number of elements: ");
44     scanf("%d", &n);
45     printf("Enter the array elements:\n");
46     for(i = 0; i < n; i++)
47     {
48         scanf("%d", &arr[i]);
49     }
50     mergeSort(arr, 0, n - 1);
51     printf("\nSorted Array: ");
52     for(i = 0; i < n; i++)
53     {
54         printf("%d ", arr[i]);
55     }
56     return 0;
57 }
```

### Sample input and output:

```
Enter the number of elements: 6
Enter the array elements:
5 2 3 1 4 6

Sorted Array: 1 2 3 4 5 6
```

## Time Complexity

- Best Case:  $O(n \log n)$
- Average Case:  $O(n \log n)$
- Worst Case:  $O(n \log n)$

## Space Complexity

- $O(n)$  (Auxiliary array used during merging)

## Result

Thus, the C program to sort an array in ascending order using Merge Sort was implemented successfully. The program sorts the array without using built-in functions and achieves the required  $O(n \log n)$  time complexity.

**11. Given an  $m \times n$  grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly  $N$  steps. Example: Input:  $m=2, n=2, N=2, i=0, j=0$  Input:  $m=1, n=3, N=3, i=0, j=1$  Output: 6, Output: 12**

## Aim

To write a C program to find the **number of ways** to move a ball out of an  $m \times n$  grid in exactly  $N$  steps, starting from a given cell  $(i, j)$ .

## Algorithm

1. Start the program.
2. Read the values of  $m$ ,  $n$ ,  $N$ , and the starting position  $(i, j)$ .
3. Define four possible directions:
  - Up
  - Down
  - Left
  - Right
4. Use recursion to explore all possible moves.
5. If the ball moves outside the grid exactly after  $N$  steps, count it as one valid path.
6. If the ball moves outside before or after  $N$  steps, do not count it.
7. Sum all valid paths.
8. Display the total number of ways.
9. Stop the program.

## Source Code:

```
1  #include <stdio.h>
2
3  int countWays(int m, int n, int N, int i, int j)
4  {
5      if(i < 0 || i >= m || j < 0 || j >= n)
6      {
7          if(N == 0)
8              return 1;
9          else
10             return 0;
11     }
12     if(N == 0)
13         return 0;
14     return countWays(m, n, N - 1, i - 1, j) + // Up
15            countWays(m, n, N - 1, i + 1, j) + // Down
16            countWays(m, n, N - 1, i, j - 1) + // Left
17            countWays(m, n, N - 1, i, j + 1); // Right
18 }

16     countWays(m, n, N - 1, i, j - 1) + // Left
17     countWays(m, n, N - 1, i, j + 1); // Right
18 }
19 int main()
20 {
21     int m, n, N, i, j;
22     printf("Enter number of rows: ");
23     scanf("%d", &m);
24     printf("Enter number of columns: ");
25     scanf("%d", &n);
26     printf("Enter number of steps: ");
27     scanf("%d", &N);
28     printf("Enter starting row and column: ");
29     scanf("%d%d", &i, &j);
30     printf("\nNumber of Ways = %d\n", countWays(m, n, N, i, j));
31     return 0;
32 }
```

## Sample input and output:

```
Enter number of rows: 2
Enter number of columns: 2
Enter number of steps:
2
Enter starting row and column: 0 0

Number of Ways = 4
```

## Time Complexity

Each step has 4 possible moves (up, down, left, right).

Time Complexity =  $O(4^N)$

## Space Complexity

The maximum recursion depth is N.

Space Complexity =  $O(N)$

## Result

Thus, the C program to find the number of ways to move the ball out of an  $m \times n$  grid in exactly N steps was implemented successfully. The program correctly computes the number of valid paths for the given test cases using recursion.

**12. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night. Examples: Input : nums = [2, 3, 2] Output : The maximum money you can rob without alerting the police is 3(robbing house 1). (ii) Input : nums = [1, 2, 3, 1] Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).**

#### **Aim:**

To write a C program to find the maximum amount of money that can be robbed from houses arranged in a circle without robbing two adjacent houses.

#### **Algorithm:**

1. Start the program.
2. Read the number of houses (n).
3. Read the amount of money in each house.
4. If there is only one house, return its value.
5. Since the houses are arranged in a circle, solve two cases:
  - Case 1: Rob houses from index 0 to n-2 (exclude the last house).
  - Case 2: Rob houses from index 1 to n-1 (exclude the first house).
6. For each case, use Dynamic Programming:
  - Maintain two variables:
    - prev1 = Maximum money till the previous house.
    - prev2 = Maximum money till the house before the previous one.
  - For each house, calculate:  
current = max(prev1, prev2 + money)
7. The answer is the maximum of the two cases.
8. Display the maximum money.
9. Stop the program.

#### **Source Code:**

```
1  #include <stdio.h>
2  int max(int a, int b)
3  {
4      return (a > b) ? a : b;
5  }
6  int robLinear(int arr[], int start, int end)
7  {
8      int prev1 = 0, prev2 = 0, current;
9      for(int i = start; i <= end; i++)
10     {
11         current = max(prev1, prev2 + arr[i]);
12         prev2 = prev1;
13         prev1 = current;
14     }
15     return prev1;
16 }
17 int main()
18 {
```

```
19     int n;
20     int nums[100];
21     printf("Enter number of houses: ");
22     scanf("%d", &n);
23     printf("Enter money in each house:\n");
24     for(int i = 0; i < n; i++)
25     {
26         scanf("%d", &nums[i]);
27     }
28     if(n == 1)
29     {
30         printf("\nMaximum Money = %d\n", nums[0]);
31         return 0;
32     }
33     int case1 = robLinear(nums, 0, n - 2);
34     int case2 = robLinear(nums, 1, n - 1);
35     printf("\nMaximum Money = %d\n", max(case1, case2));
36     return 0;
```

### Sample input and output:

```
Enter number of houses: 3
Enter money in each house:
2 3 2
```

```
Maximum Money = 3
```

```
Enter number of houses: 4
Enter money in each house:
1 2 3 1
```

```
Maximum Money = 4
```

### Time Complexity

The array is traversed twice (once excluding the last house and once excluding the first house).

Time Complexity =  $O(n)$

### Space Complexity

Only a few variables are used for Dynamic Programming.

Space Complexity =  $O(1)$

### Result

Thus, the C program to solve the House Robber II problem was implemented successfully. The program correctly finds the maximum amount of money that can be robbed from houses arranged in a circular street without robbing two adjacent houses. The algorithm runs in  $O(n)$  time and uses  $O(1)$  extra space.

**13. You are climbing a staircase. It takes  $n$  steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top? Examples: Input:  $n=4$  Output: 5 Input:  $n=3$  Output: 3**

### Aim

To write a C program to find the number of distinct ways to climb a staircase of  $n$  steps, where at each step you can climb either 1 step or 2 steps.

### Algorithm

1. Start the program.
2. Read the number of steps ( $n$ ).
3. If  $n = 1$ , return 1.
4. If  $n = 2$ , return 2.
5. Initialize:
  - first = 1 (ways to reach step 1)
  - second = 2 (ways to reach step 2)
6. For each step from 3 to  $n$ :

- `current = first + second`
  - Update:
    - `first = second`
    - `second = current`
7. Display the value of current (or second).
  8. Stop the program.

### Source Code:

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int n;
6      int first = 1, second = 2, current;
7
8      printf("Enter the number of steps: ");
9      scanf("%d", &n);
10
11     if(n == 1)
12     {
13         printf("Number of Ways = 1\n");
14         return 0;
15     }
16
17     if(n == 2)
18     {

```

```

16
17     if(n == 2)
18     {
19         printf("Number of Ways = 2\n");
20         return 0;
21     }
22
23     for(int i = 3; i <= n; i++)
24     {
25         current = first + second;
26         first = second;
27         second = current;
28     }
29
30     printf("Number of Ways = %d\n", second);
31
32     return 0;
33 }

```

### Sample input and output:

```

Enter the number of steps: 4
Number of Ways = 5

```

```

Enter the number of steps: 3
Number of Ways = 3

```

### Time Complexity

The algorithm traverses the steps only once.

Time Complexity =  $O(n)$

### Space Complexity

Only three integer variables are used (first, second, and current).

Space Complexity =  $O(1)$

### Result

Thus, the C program to find the number of distinct ways to climb a staircase was implemented successfully. The program correctly computes the number of possible ways using Dynamic Programming with  $O(n)$  time complexity and  $O(1)$  space complexity.

**14. A robot is located at the top-left corner of a  $m \times n$  grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there? Examples: Input:  $m=7,n=3$      Input:  $m=3,n=2$   
Output: 28     Output: 3**

### Aim

To write a C program to find the number of unique paths for a robot to move from the top-left corner to the bottom-right corner of an  $m \times n$  grid, where the robot can move only right or down.

### Algorithm

1. Start the program.
2. Read the number of rows (m) and columns (n).
3. Create a DP (Dynamic Programming) table  $dp[m][n]$ .
4. Initialize the first row and first column with 1, since there is only one way to reach those cells.
5. For each remaining cell:
  - Calculate:

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$

6. The value in  $dp[m-1][n-1]$  gives the total number of unique paths.
7. Display the result.
8. Stop the program.

### Source Code:

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int m, n;
6      int dp[100][100];
7
8      printf("Enter number of rows: ");
9      scanf("%d", &m);
10
11     printf("Enter number of columns: ");
12     scanf("%d", &n);
13
14     // Initialize first row and first column
15     for(int i = 0; i < m; i++)
16         dp[i][0] = 1;
17
18     for(int j = 0; j < n; j++)
19         dp[0][j] = 1;
20
21     // Fill the remaining cells
22     for(int i = 1; i < m; i++)
23     {
24         for(int j = 1; j < n; j++)
25         {
26             dp[i][j] = dp[i-1][j] + dp[i][j-1];
27         }
28     }
29
30     printf("\nNumber of Unique Paths = %d\n", dp[m-1][n-1]);
31
32     return 0;
33 }
```

### Sample input and output:

```

Enter number of rows: 7
Enter number of columns: 3

Number of Unique Paths = 28
```

```

Enter number of rows: 3
Enter number of columns: 2

Number of Unique Paths = 3
```

### Time Complexity:

The DP table is filled once.

Time Complexity =  $O(m \times n)$

### Space Complexity:

The DP table stores  $m \times n$  values.

Space Complexity =  $O(m \times n)$

### Result

Thus, the C program to find the number of unique paths in an  $m \times n$  grid was implemented successfully. The program correctly calculates the total number of paths using Dynamic Programming. The algorithm has a time complexity of  $O(m \times n)$  and a space complexity of  $O(m \times n)$ .

**15. In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzzy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index. Example 1: Input: s = "abbxxxxzzy" Output: [[3,6]] Explanation: "xxxx" is the only large group with start index 3 and end index 6. Example 2: Input: s = "abc" Output: [] Explanation: We have groups "a", "b", and "c", none of which are large groups.**

### Aim

To write a C program to identify all **large groups** (groups of 3 or more consecutive identical characters) in a given string and print their **start** and **end** indices.

### Algorithm

1. Start the program.
2. Read the input string.
3. Initialize start = 0.
4. Traverse the string from left to right.
5. Whenever the current character is different from the next character (or the end of the string is reached):
  - Calculate the group length:

length = end - start + 1

- If the length is **3 or more**, print the interval [start, end].
  - Update start to the next index.
6. If no large group is found, print an empty list [].



7. Stop the program.

### Source Code:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char s[100];
7     int start = 0;
8     int found = 0;
9
10    printf("Enter the string: ");
11    scanf("%s", s);
12
13    int n = strlen(s);
14
15    for(int i = 0; i < n; i++)
16    {
17        if(i == n - 1 || s[i] != s[i + 1])
18        {
19            if(i - start + 1 >= 3)
20            {
21                printf("[%d,%d] ", start, i);
22                found = 1;
23            }
24            start = i + 1;
25        }
26    }
27
28    if(found == 0)
29    {
30        printf("[]");
31    }
32
33    printf("\n");
34
35    return 0;
36 }
```

### Sample input and output:

```
Enter the string: abbxxxxzy
[3,6]
```

```
Enter the string: 123
[]
```

### Time Complexity

The string is traversed only once.

Time Complexity =  $O(n)$

where  $n$  is the length of the string.

### Space Complexity

The program uses only a few extra variables.

Space Complexity =  $O(1)$

### Result

Thus, the C program to find all large groups (three or more consecutive identical characters) in a string was implemented successfully. The program correctly prints the intervals of all large groups in increasing order of their starting index.

16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an  $m \times n$  grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules. Any live cell with fewer than two live neighbors dies as if caused by under-population. Any live cell with two or three live neighbors lives on to the next generation. Any live cell with more than three live neighbors dies, as if by over-population. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction. The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the  $m \times n$  grid board, return the next state. Example 1: Input: board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]] Output: [[0,0,0],[1,0,1],[0,1,1],[0,1,0]] Example 2: Input: board = [[1,1],[1,0]] Output: [[1,1],[1,1]]

### Aim

To write a C program to determine the next state of an  $m \times n$  grid according to the rules of Conway's Game of Life.

### Algorithm

1. Start the program.
2. Read the number of rows (m) and columns (n).
3. Read the current state of the board.
4. For each cell:
  - Count the number of live neighbors among the eight surrounding cells.
  - Apply the following rules:
    - If the cell is live and has fewer than 2 live neighbors, it dies.
    - If the cell is live and has 2 or 3 live neighbors, it remains alive.
    - If the cell is live and has more than 3 live neighbors, it dies.
    - If the cell is dead and has exactly 3 live neighbors, it becomes alive.
5. Store the updated state in a temporary array.
6. Copy the temporary array to the original board.
7. Display the next state of the board.
8. Stop the program.

### Source Code:

```
1 #include <stdio.h>
2 int main()
3 {
4     int m, n;
5     int board[100][100], next[100][100];
6     printf("Enter number of rows: ");
7     scanf("%d", &m);
8     printf("Enter number of columns: ");
9     scanf("%d", &n);
10    printf("Enter the board elements (0 or 1):\n");
11    for(int i = 0; i < m; i++)
12    {
13        for(int j = 0; j < n; j++)
14        {
15            scanf("%d", &board[i][j]);
16        }
17    }
18    for(int i = 0; i < m; i++)
```

```
20    for(int j = 0; j < n; j++)
21    {
22        int live = 0;
23        for(int x = -1; x <= 1; x++)
24        {
25            for(int y = -1; y <= 1; y++)
26            {
27                if(x == 0 && y == 0)
28                    continue;
29                int r = i + x;
30                int c = j + y;
31                if(r >= 0 && r < m && c >= 0 && c < n)
32                {
33                    if(board[r][c] == 1)
34                        live++;
35                }
36            }
37        }
```

```
46    }
47    else
48    {
49        if(live == 3)
50            next[i][j] = 1;
51        else
52            next[i][j] = 0;
53    }
54 }
55 }
56 printf("\nNext State of the Board:\n");
57 for(int i = 0; i < m; i++)
58 {
59     for(int j = 0; j < n; j++)
60     {
61         printf("%d ", next[i][j]);
62     }
63     printf("\n");
```

### Sample input and output:

```
Enter number of rows: 4
Enter number of columns: 3
Enter the board elements (0 or 1):
0 1 0
0 0 1
1 1 1
0 0 0

Next State of the Board:
0 0 0
1 0 1
0 1 1
0 1 0
```

```
Enter number of rows: 2
Enter number of columns: 2
Enter the board elements (0 or 1):
1 1
1 0

Next State of the Board:
1 1
1 1
```

### Time Complexity

Each cell is visited once, and for every cell, its 8 neighbors are checked.

Time Complexity =  $O(m \times n)$

### Space Complexity

A temporary matrix of size  $m \times n$  is used to store the next state.

Space Complexity =  $O(m \times n)$

### Result

Thus, the C program to implement Conway's Game of Life was executed successfully. The program correctly computes the next generation of the board by applying the four Game of Life rules simultaneously to every cell.

**17. We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below. Now after pouring some non-negative integer cups of champagne, return how full the  $j$ th glass in the  $i$ th row is (both  $i$  and  $j$  are 0-indexed.)** Example 1: Input: poured = 1, query\_row = 1, query\_glass = 1 Output: 0.00000 Explanation: We poured 1 cup of champagne to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty. Example 2: Input: poured = 2, query\_row = 1, query\_glass = 1 Output: 0.50000 Explanation: We poured 2 cups of champagne to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champagne.

## Aim

To write a C program to determine how full a specific glass is in a Champagne Tower after pouring a given number of cups of champagne.

## Algorithm

1. Start the program.
2. Read the values of:
  - poured
  - query\_row
  - query\_glass
3. Create a 2D array tower[101][101] and initialize all elements to 0.
4. Pour the champagne into the top glass:

tower[0][0] = poured

5. Traverse each row up to query\_row:
  - If a glass contains more than 1 cup, calculate the excess:

excess = (tower[i][j] - 1) / 2

- Add the excess equally to the two glasses below:
    - Left child: tower[i+1][j]
    - Right child: tower[i+1][j+1]
6. The amount in the queried glass is:
    - 1.0 if it contains more than one cup.
    - Otherwise, its current value.
  7. Display the result.
  8. Stop the program.

## Source Code:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int poured, query_row, query_glass;
6     double tower[101][101] = {0};
7     printf("Enter poured cups: ");
8     scanf("%d", &poured);
9     printf("Enter query row: ");
10    scanf("%d", &query_row);
11    printf("Enter query glass: ");
12    scanf("%d", &query_glass);
13    tower[0][0] = poured;
14    for(int i = 0; i < query_row; i++)
15    {
16        for(int j = 0; j <= i; j++)
17        {
18            if(tower[i][j] > 1.0)
```

```
16        for(int j = 0; j <= i; j++)
17        {
18            if(tower[i][j] > 1.0)
19            {
20                double excess = (tower[i][j] - 1.0) / 2.0;
21                tower[i + 1][j] += excess;
22                tower[i + 1][j + 1] += excess;
23                tower[i][j] = 1.0;
24            }
25        }
26    }
27
28    if(tower[query_row][query_glass] > 1.0)
29        tower[query_row][query_glass] = 1.0;
30
31    printf("\nGlass Fullness = %.5lf\n", tower[query_row][query_glass]);
32    return 0;
33 }
```

### Sample input and output:

```
Enter poured cups: 1
Enter query row: 1
Enter query glass: 1

Glass Fullness = 0.00000
```

```
Enter poured cups: 2
Enter query row: 1
Enter query glass: 1

Glass Fullness = 0.50000
```

### Time Complexity

The algorithm processes each glass up to the queried row.

Time Complexity =  $O(\text{query\_row}^2)$

### Space Complexity

A 2D array of size  $101 \times 101$  is used.

Space Complexity =  $O(101^2)$  (constant space, since the tower size is fixed), or generally  $O(\text{query\_row}^2)$ .

### Result

Thus, the C program to determine the fullness of a glass in a Champagne Tower was implemented successfully. The program correctly computes the amount of champagne in the specified glass after distributing the excess liquid according to the given rules.